

Aritmética do computador



Objetivos de aprendizagem

Após ler este capítulo, você será capaz de:

- Fazer a distinção entre a maneira como números são representados (formato binário) e os algoritmos usados para as operações aritméticas básicas.
- Explicar representação em complemento de dois.
- Apresentar uma visão geral das técnicas para a operação aritmética básica em notação de complemento de dois.

Objetivos de aprendizagem

WILLIAM STALLINGS

**Arquitetura e
Organização de
Computadores**

10ª edição

Após ler este capítulo, você será capaz de:

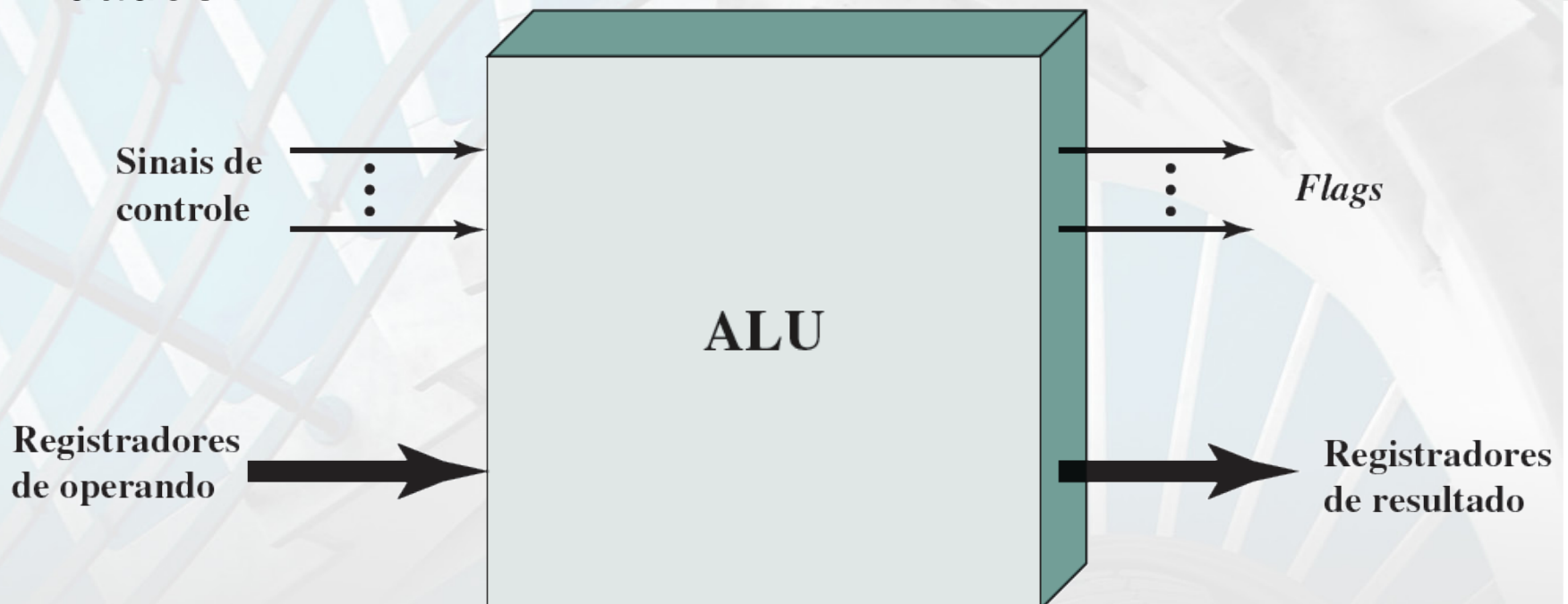
- Compreender o uso de significando, base e expoente na representação de números de ponto flutuante.
- Apresentar uma visão geral do padrão IEEE 754 para a representação em ponto flutuante.
- Entender alguns conceitos-chave relacionados à aritmética de ponto flutuante, inclusive bits de guarda, arredondamento, números subnormais, *underflow* e *overflow*.

Unidade Lógica e Aritmética (ALU)

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- A **ALU** é a parte do computador que realmente realiza operações lógicas e aritméticas sobre os dados:



Representação em inteiros

- Para os propósitos de armazenamento e processamento no computador, somente dígitos binários (0 e 1) podem ser usados para representar os números.
- Se estivermos limitados a inteiros não negativos, a representação é direta.

Uma palavra de 8 bits pode representar os números de 0 a 255, incluindo

$$A = \sum_{i=0}^{n-1} 2^i a_i$$

00000000 = 0

00000001 = 1

00101001 = 41

10000000 = 128

11111111 = 255

Representação em sinal-magnitude

- A forma de representação mais simples que emprega um bit de sinal é a **representação sinal-magnitude**.
- Em uma palavra de n bits, os $n - 1$ bits mais à direita representam a magnitude do inteiro.

$$+18 = 00010010$$

$$-18 = 10010010 \quad (\text{sinal-magnitude})$$

- O caso geral pode ser expresso da seguinte forma:

Sinal-magnitude

$$A = \begin{cases} \sum_{i=0}^{n-2} 2^i a_i & \text{se } a_{n-1} = 0 \\ -\sum_{i=0}^{n-2} 2^i a_i & \text{se } a_{n-1} = 1 \end{cases}$$

Representação em complemento de dois

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Características da representação e aritmética de complemento de dois:

Intervalo	-2^{n-1} a $2^{n-1} - 1$
Número de representações de zero	Um
Negação	Apanhe o complemento booleano de cada bit do número positivo correspondente, depois some 1 ao padrão de bits resultante visto como um inteiro sem sinal.
Expansão da extensão em bits	Acrescente posições de bit adicionais à esquerda e preencha com o valor do bit de sinal original.
Regra de <i>overflow</i>	Se dois números com o mesmo sinal (positivo ou negativo) são somados, então o <i>overflow</i> ocorre se e somente se o resultado tem o sinal oposto.
Regra de subtração	Para subtrair B de A , pegue o complemento de dois de B e some-o a A .

Representação em complemento de dois

- Considere um inteiro de n bits, A , na representação em complemento de dois.
- Se A for positivo, então o bit de sinal, a_{n-1} , é zero.
- Os bits restantes representam a magnitude do número da mesma forma que a representação sinal-magnitude:

$$A = \sum_{i=0}^{n-2} 2^i a_i \quad \text{para } A \geq 0$$

- Expressão para números negativos:

Complemento de dois

$$A = -2^{n-1} a_{n-1} + \sum_{i=0}^{n-2} 2^i a_i$$

Representação em complemento de dois

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Uso de uma caixa de valores para a conversão entre complemento de dois e decimal.
- Uma caixa de valores de complemento de dois com oito posições:

-128	64	32	16	8	4	2	1

Representação em complemento de dois

- Uso de uma caixa de valores para a conversão entre complemento de dois e decimal.
- Conversão do binário 10000011 para o decimal:

-128	64	32	16	8	4	2	1
1	0	0	0	0	0	1	1

-128

+2 +1 = -125

Representação em complemento de dois

- Uso de uma caixa de valores para a conversão entre complemento de dois e decimal.
- Conversão do decimal -120 para o binário:

-128	64	32	16	8	4	2	1
1	0	0	0	1	0	0	0

-120 = -128

+8

Aritmética com inteiros

- Na representação sinal-magnitude, a regra para formar a **negação** de um inteiro é simples: inverta o bit de sinal.
- Na notação de complemento de dois, a negação de um inteiro pode ser formada com as seguintes regras:
 1. Apanhe o complemento booleano de cada bit do inteiro. Ou seja, defina cada 1 como 0, e cada 0 como 1.
 2. Trate o resultado como um inteiro binário sem sinal, some 1.
- Esse processo em duas etapas é conhecido como a **operação de complemento de dois**.

Aritmética com inteiros

- **Adição** de números na representação em complemento de dois:

$\begin{array}{r} 1001 = -7 \\ + \underline{0101} = 5 \\ 1110 = -2 \\ \text{(a) } (-7) + (+5) \end{array}$	$\begin{array}{r} 1100 = -4 \\ + \underline{0100} = 4 \\ \underline{1}0000 = 0 \\ \text{(b) } (-4) + (+4) \end{array}$
$\begin{array}{r} 0011 = 3 \\ + \underline{0100} = 4 \\ 0111 = 7 \\ \text{(c) } (+3) + (+4) \end{array}$	$\begin{array}{r} 1100 = -4 \\ + \underline{1111} = -1 \\ \underline{1}1011 = -5 \\ \text{(d) } (-4) + (-1) \end{array}$
$\begin{array}{r} 0101 = 5 \\ + \underline{0100} = 4 \\ 1001 = \text{Overflow} \\ \text{(e) } (+5) + (+4) \end{array}$	$\begin{array}{r} 1001 = -7 \\ + \underline{1010} = -6 \\ \underline{1}0011 = \text{Overflow} \\ \text{(f) } (-7) + (-6) \end{array}$

Aritmética com inteiros

- Em qualquer adição, o resultado pode ser maior do que pode ser mantido no tamanho utilizado da palavra.
- Essa condição é chamada de ***overflow***.
- **Regra do overflow:** se dois números são somados e ambos são positivos ou ambos são negativos, então o overflow ocorre se, e somente se, o resultado tiver o sinal oposto.
- **Regra da subtração:** para subtrair um número (subtraendo) de outro (minuendo), pegue o complemento de dois (negação) do subtraendo e some-o ao minuendo.

Aritmética com inteiros

- Em comparação com a adição e a subtração, a **multiplicação** é uma operação complexa, seja ela realizada no hardware ou pelo software.
- Vários algoritmos foram usados em diversos computadores.
- Multiplicação de inteiros binários sem sinal:

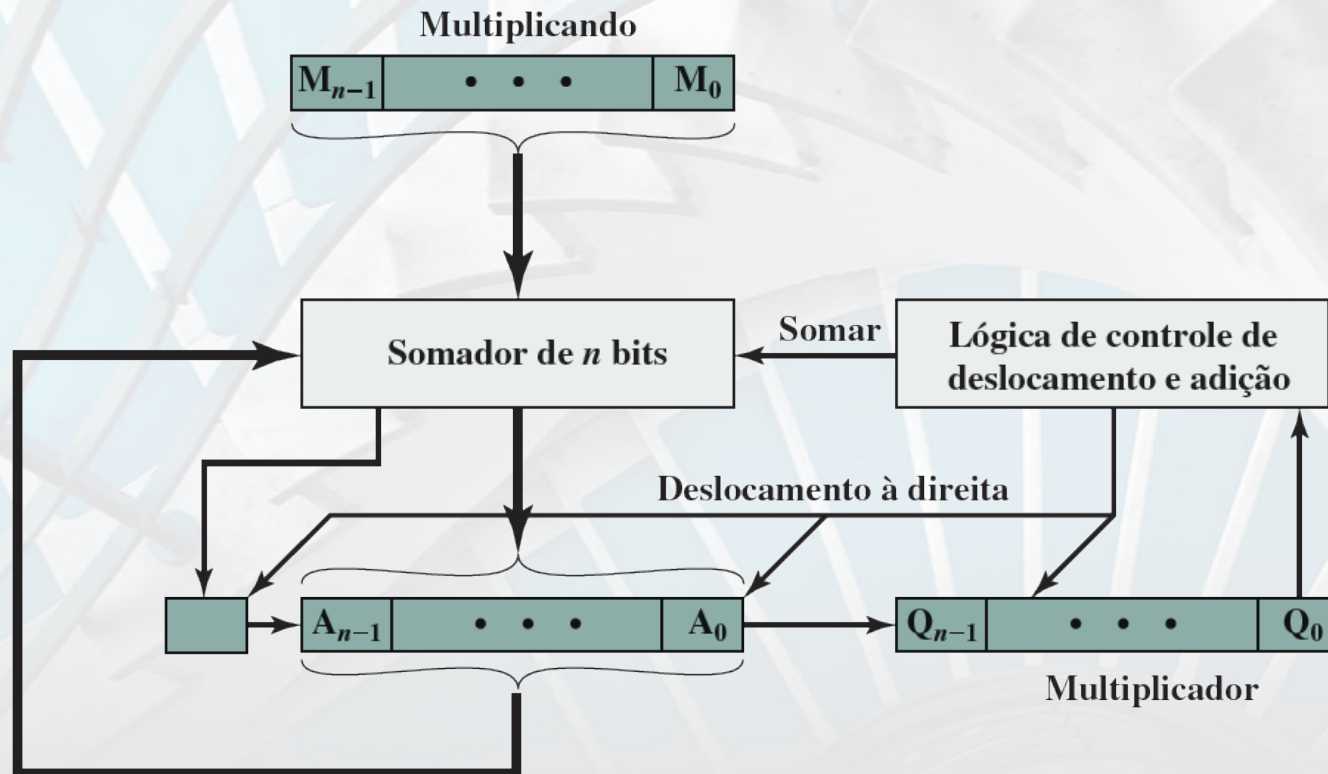
1011		Multiplicando (11)
×1101		Multiplicador (13)
1011	}	Produtos parciais
0000		
1011		
1011		
10001111		Produto (143)

Aritmética com inteiros

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Implementação de hardware da multiplicação binária sem sinal:



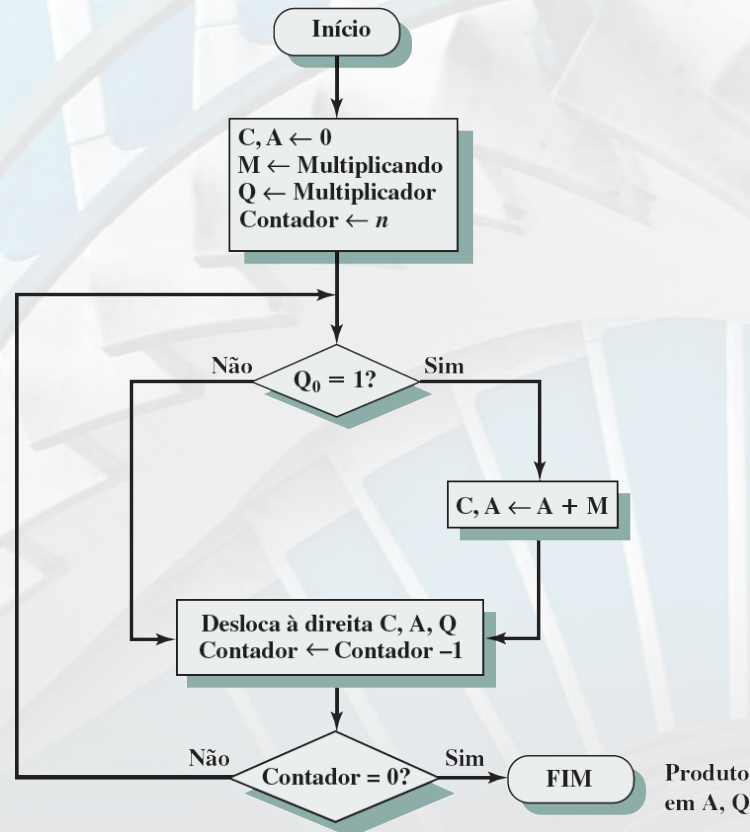
Aritmética com inteiros

WILLIAM STALLINGS

Arquitetura e Organização de Computadores

10ª edição

- Fluxograma para a multiplicação binária sem sinal:



Aritmética com inteiros

- Multiplicação de dois inteiros de 4 bits sem sinal, gerando um resultado de 8 bits:

1011	
× 1101	
00001011	$1011 \times 1 \times 2^0$
00000000	$1011 \times 0 \times 2^1$
00101100	$1011 \times 1 \times 2^2$
01011000	$1011 \times 1 \times 2^3$
10001111	

Aritmética com inteiros

- Comparação da multiplicação de inteiros sem sinal e em complemento de dois:

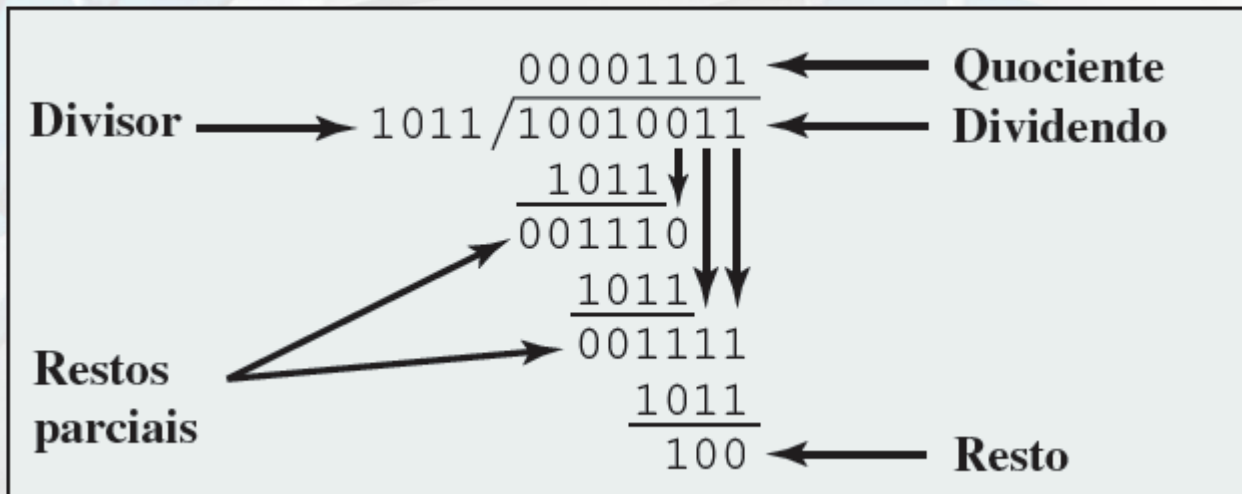
$ \begin{array}{r} 1001 \quad (9) \\ \times 0011 \quad (3) \\ \hline 00001001 \quad 1001 \times 2^0 \\ 00010010 \quad 1001 \times 2^1 \\ \hline 00011011 \quad (27) \end{array} $	$ \begin{array}{r} 1001 \quad (-7) \\ \times 0011 \quad (3) \\ \hline 11111001 \quad (-7) \times 2^0 = (-7) \\ 11110010 \quad (-7) \times 2^1 = (-14) \\ \hline 11101011 \quad (-21) \end{array} $
---	--

Inteiros sem sinal

Inteiros em complemento de dois

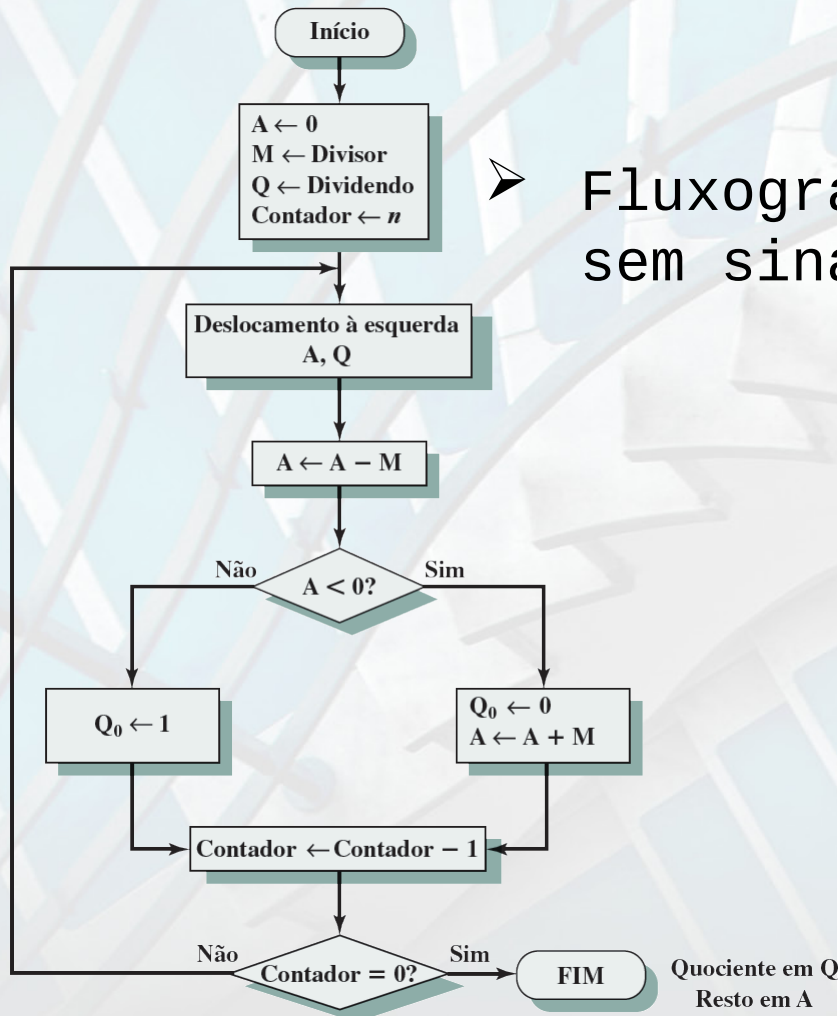
Aritmética com inteiros

- A **divisão** é um pouco mais complexa que a multiplicação, mas é baseada nos mesmos princípios gerais.
- Exemplo de divisão de inteiros binários sem sinal:



Aritmética com inteiros

➤ Fluxograma para divisão binária sem sinal:



Aritmética com inteiros

WILLIAM STALLINGS

Arquitetura e Organização de Computadores

10ª edição

- Exemplo de divisão por restauração em complemento de dois (7/3):

A	Q	
0000	0111	Valor inicial
0000 <u>1101</u> 1101	1110	Deslocamento Use dois complementos de 0011 para a subtração Subtraia
0000	1110	Restaure, faça $Q_0 = 0$
0001 <u>1101</u> 1110	1100	Deslocamento Subtraia
0001	1100	Restaure, faça $Q_0 = 0$
0011 <u>1101</u> 0000	1000	Deslocamento
	1001	Restaure, faça $Q_0 = 1$
0001 <u>1101</u> 1110	0010	Deslocamento Subtraia
0001	0010	Restaure, faça $Q_0 = 0$

Representação em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Podemos representar um número no formato

$$\pm S \times B^{\pm E}$$

- Esse número pode ser armazenado em uma palavra binária com três campos:

1. Sinal: mais ou menos.
2. Significando S.
3. Expoente E.

- A **base** é implícita e não precisa ser armazenada, pois é a mesma para todos os números.

Representação em ponto flutuante

- Abaixo vemos um formato típico de ponto flutuante com 32 bits.
- O bit mais à esquerda armazena o **sinal** do número (0 = positivo, 1 = negativo).
- O valor do **expoente** é armazenado nos 8 bits seguintes.
- A representação usada é conhecida como **representação polarizada** .

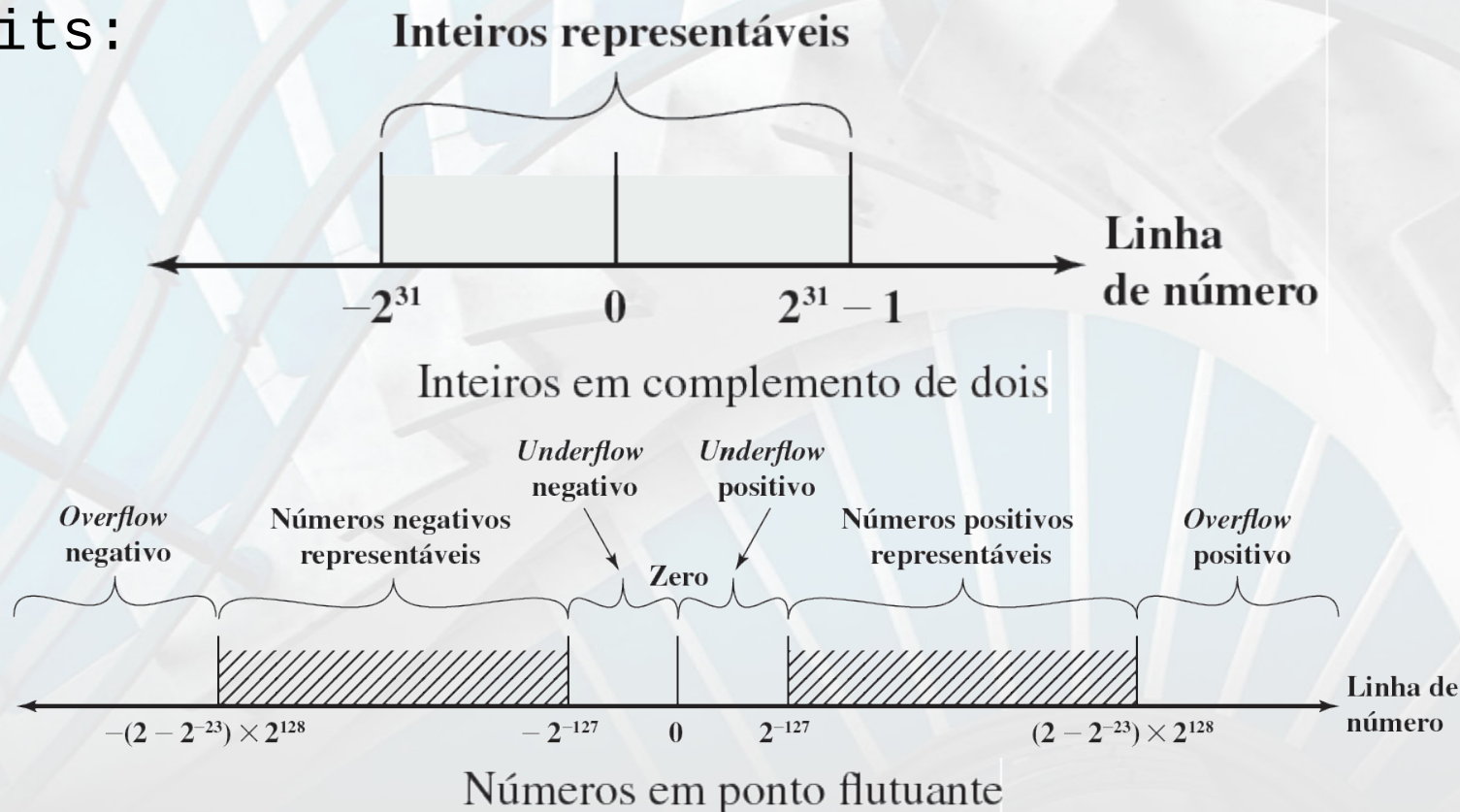


Representação em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Números expressos em formatos típicos de 32 bits:



Padrão do IEEE para a representação binária em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

O IEEE 754 de 2008 define os seguintes tipos diferentes de formatos de ponto flutuante:

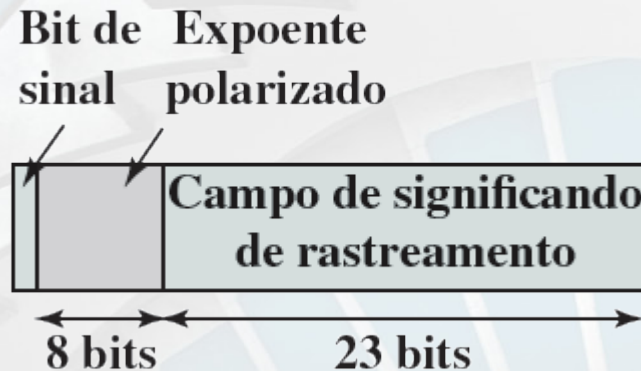
- **Formato aritmético:** Todas as operações obrigatórias definidas pelo padrão são aceitas pelo formato.
- **Formato básico:** Cobre cinco representações de ponto flutuante, três binárias e duas decimais, cujas codificações são especificadas pelo padrão e podem ser usadas pela aritmética.
- **Formato de intercâmbio:** Uma codificação de tamanho fixo completamente especificada que possibilita a troca de dados entre plataformas distintas.

Padrão do IEEE para a representação binária em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Formatos do IEEE 754:
- Formato binário32

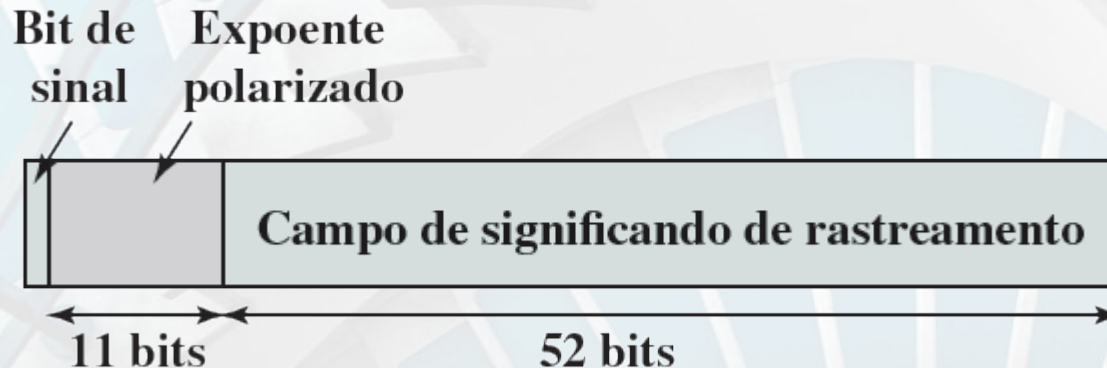


Padrão do IEEE para a representação binária em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Formatos do IEEE 754:
- Formato binário64



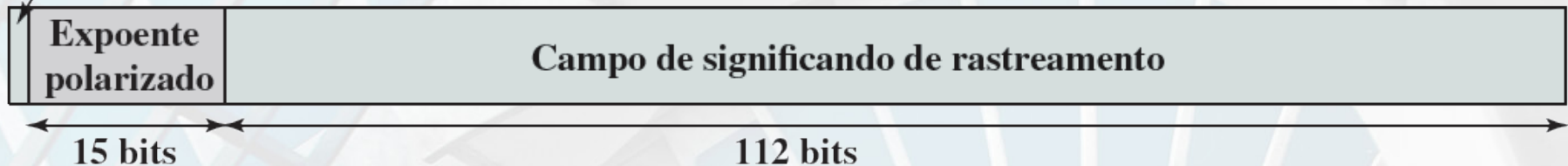
Padrão do IEEE para a representação binária em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Formatos do IEEE 754:
- Formato binário128

Bit de sinal



Padrão do IEEE para a representação binária em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

➤ Formatos de IEEE:

Formato	Tipo de formato		
	Formato aritmético	Formato básico	Formato de intercâmbio
binário16			X
binário32	X	X	X
binário64	X	X	X
binário128	X	X	X
binário{k} ($k = n \times 32$ para $n > 4$)	X		X
decimal64	X	X	X
decimal128	X	X	X
decimal{k} ($k = n \times 32$ para $n > 4$)	X		X
precisão estendida	X		
precisão extensível	X		

Aritmética em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Números e operações aritméticas de ponto flutuante:

Números de ponto flutuante

$$X = X_S \times B^{X_E}$$
$$Y = Y_S \times B^{Y_E}$$

Operações aritméticas

$$\left. \begin{aligned} X + Y &= (X_S \times B^{X_E - Y_E} + Y_S) \times B^{Y_E} \\ X - Y &= (X_S \times B^{X_E - Y_E} - Y_S) \times B^{Y_E} \end{aligned} \right\} X_E \leq Y_E$$
$$X \times Y = (X_S \times Y_S) \times B^{X_E + Y_E}$$
$$\frac{X}{Y} = \left(\frac{X_S}{Y_S} \right) \times B^{X_E - Y_E}$$

Aritmética em ponto flutuante

WILLIAM STALLINGS
Arquitetura e
Organização de
Computadores

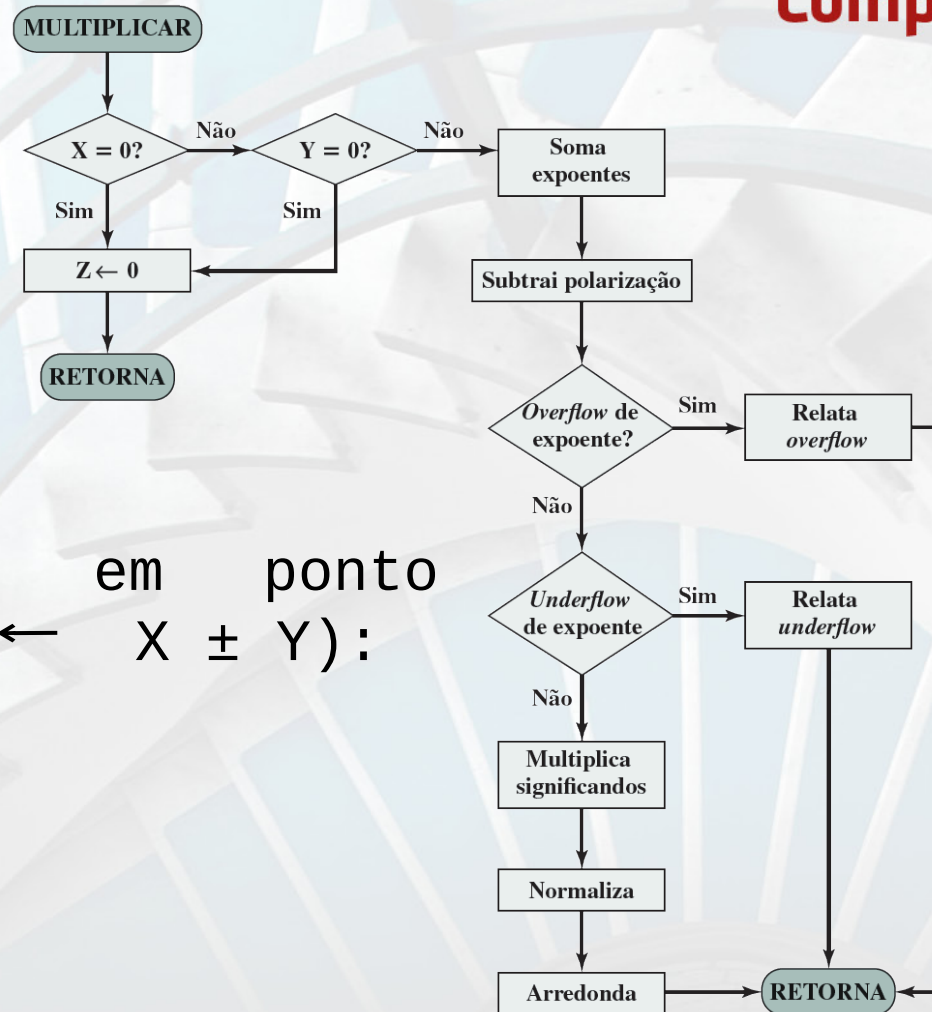
10ª edição

- Na aritmética de ponto flutuante, adição e subtração são mais complexas do que multiplicação e divisão.
- Isso se deve à necessidade de alinhamento.
- Existem quatro fases básicas do algoritmo para adição e subtração:
 1. Verificar os zeros.
 2. Alinhar os significandos.
 3. Somar ou subtrair os significandos.
 4. Normalizar o resultado.

Aritmética em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

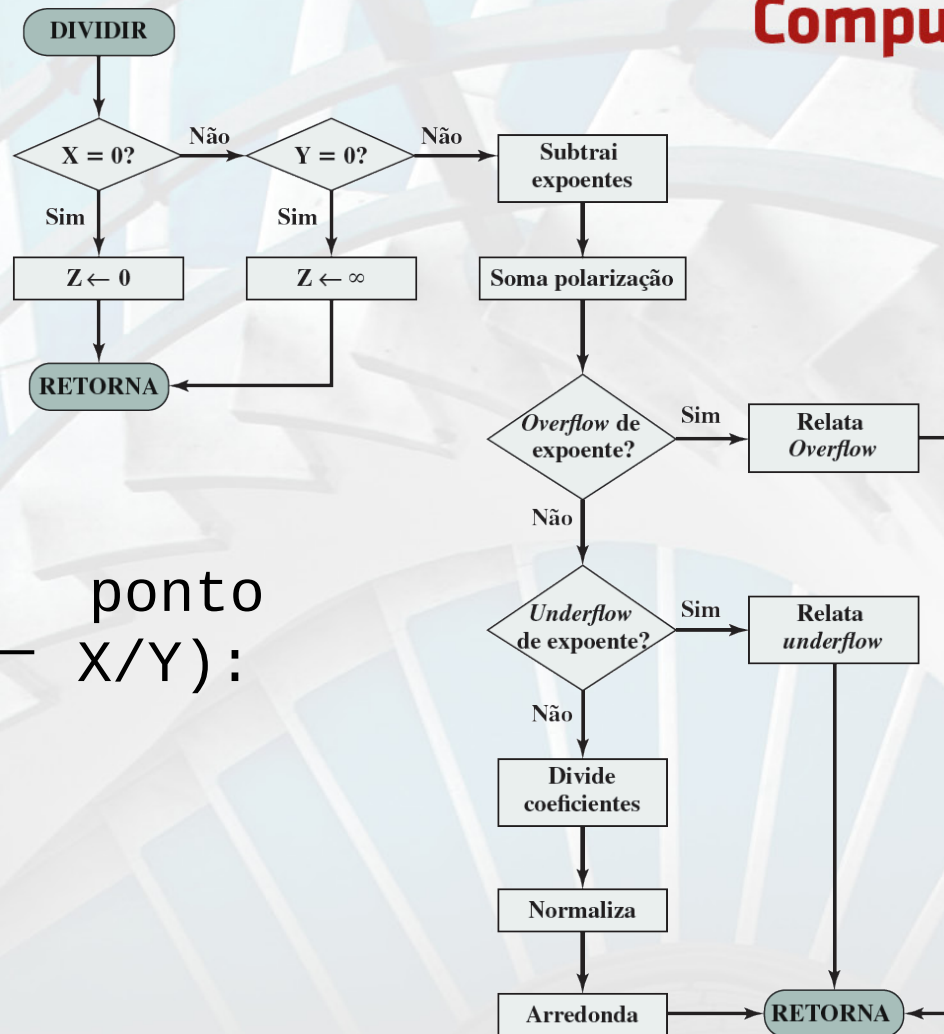


➤ Multiplicação em ponto flutuante ($Z \leftarrow X \pm Y$):

Aritmética em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição



➤ Divisão em ponto flutuante ($Z \leftarrow X/Y$):

Padrão do IEEE para a aritmética binária em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- **INFINITO** A aritmética de infinito é tratada como o caso limitador da aritmética real, com os valores de infinito recebendo a seguinte interpretação:
$$-\infty < (\text{cada número finito}) < +\infty$$
- Com a exceção dos casos especiais, qualquer operação aritmética envolvendo infinito gera o resultado óbvio, por exemplo:

$5 + (+\infty) = +\infty$	$5 \div (+\infty) = +0$
$5 - (+\infty) = -\infty$	$(+\infty) + (+\infty) = +\infty$
$5 + (-\infty) = -\infty$	$(-\infty) + (-\infty) = -\infty$
$5 - (-\infty) = +\infty$	$(-\infty) - (+\infty) = -\infty$
$5 \times (+\infty) = +\infty$	$(+\infty) - (-\infty) = +\infty$

Padrão do IEEE para a aritmética binária em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- **NaNs SILENCIOSOS E SINALIZADORES** Um NaN é uma entidade simbólica codificada em formato de ponto flutuante, do qual existem dois tipos: sinalizador e silencioso.
- Um **NaN sinalizador** sinaliza uma exceção de operação inválida sempre que aparece como um operando.
- Um **NaN silencioso** se propaga por quase todas as operações aritméticas sem sinalizar uma exceção.
- Os dois tipos de NaNs possuem o mesmo formato geral: um expoente com apenas uns e uma fração diferente de zero.

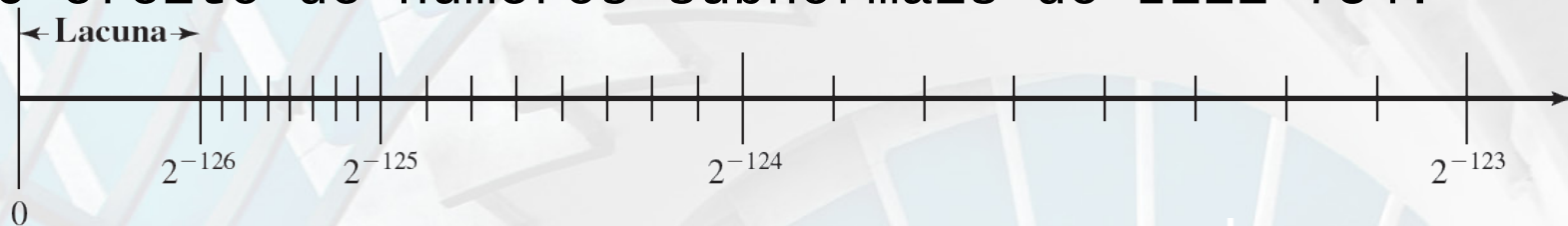
Padrão do IEEE para a aritmética binária em ponto flutuante

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

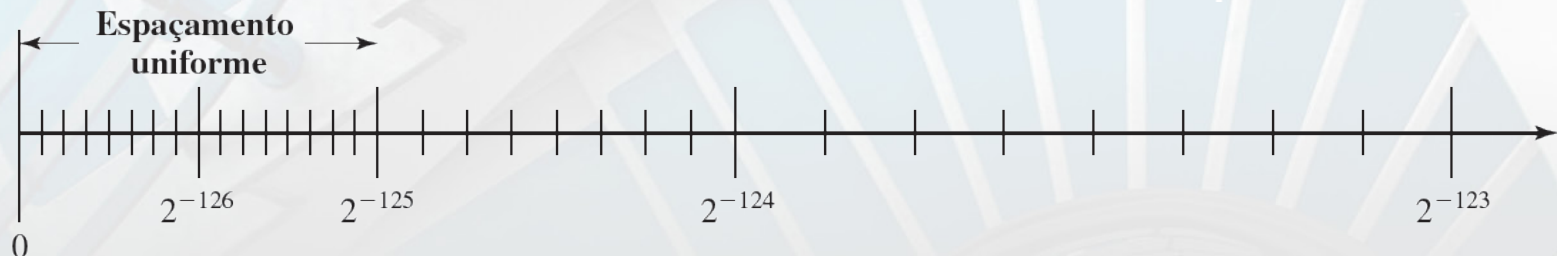
10ª edição

➤ **NÚMEROS SUBNORMAIS** Números subnormais são incluídos no IEEE 754 para lidar com casos de *underflow* de expoente.

➤ O efeito de números subnormais do IEEE 754:



Formato de 32 bits sem números subnormais



Formato de 32 bits com números subnormais